# Práctica 01

## Introducción a JavaScript: Adivina el número

Dr. José Luis Quiroz Fabián

## Índice

**1. Propósito y dinámica** **2**

**2. Preparación** **2**

**3. Primera etapa: variables y consola** **2**

**4. Segunda etapa: decisiones con prompt y alert** **3**

**5. Tercera etapa: leer el HTML** **3**

**6. Cuarta etapa: mostrar pistas en la página** **4**

**7. Quinta etapa: contar los intentos** **4**

**8. Sexta etapa: comenzar una partida nueva** **5**

**9. Séptima etapa: validar los datos** **5**

**10. Pruebas y preguntas** **6**

**11. Entrega en GitHub** **6**

**12. Material inicial proporcionado** **7**

12.1. Archivo `index.html` . . . . . 7

12.2. Archivo `style.css` . . . . . 8

12.3. Archivo `README.md` . . . . . 9

## 1. Propósito y dinámica

En esta práctica desarrollarás el comportamiento de un juego web. Recibirás los archivos `index.html`, `style.css` y `README.md` ya preparados. Tu trabajo consiste en programar `app.js`, siguiendo las instrucciones de esta guía. Al terminar, la página dará pistas sobre un número secreto y contará los intentos.

**Conocimientos que aplicarás:** variables, condicionales, funciones, `console.log()`, `alert()`, `prompt()`, `Math`, `onclick` y `document.getElementById()`.

**Explicación**

No necesitas modificar el HTML ni el CSS. Primero programarás en la consola y con ventanas emergentes; después conectarás tu programa con la interfaz web. Los primeros ejemplos son temporales y se reemplazan cuando la guía lo indique.

## 2. Preparación

**Paso 2.1** Crea una carpeta local llamada `p01_js`. Copia dentro los cuatro archivos proporcionados.

**Estructura inicial**

```
p01_js/
|-- index.html
|-- style.css
|-- app.js
`-- README.md
```

**Paso 2.2** Abre `index.html` en el navegador. La interfaz tendrá un campo numérico, dos botones, una región de mensajes y un contador.

**Paso 2.3** Abre las herramientas de desarrollo del navegador (tecla F12) y selecciona **Consola**. Por ahora, los botones no funcionan: todavía no has programado sus funciones.

**✓ Punto de control**

Comprueba que aparecen los elementos de la página y que la consola está disponible.

## 3. Primera etapa: variables y consola

**Paso 3.1** Escribe lo siguiente en `app.js`:

**app.js: primera prueba**

```
console.log("Iniciando el juego");
```

Recarga la página. El mensaje debe aparecer en la consola, no dentro de la página.

**Paso 3.2** Sustituye el contenido de `app.js` por:

**Generar el número secreto**

```
let numeroSecreto = Math.floor(Math.random() * 100) + 1;
let intentos = 0;
console.log("Numero secreto:", numeroSecreto);
```

**Explicación**

`Math.random()` produce un número entre 0 (incluido) y 1 (excluido). Al multiplicarlo por 100, aplicar `Math.floor()` y sumar 1 obtenemos un entero entre 1 y 100. `intentos` comienza en cero.

**✓ Punto de control**

Recarga varias veces: el valor secreto debe ser entero y estar entre 1 y 100.

## 4. Segunda etapa: decisiones con prompt y alert

**Paso 4.1** Conserva las dos variables y, debajo de ellas, añade temporalmente:

**Primer intento mediante prompt**

```
let numero = Number(prompt("Escribe un numero entre 1 y 100"));

if (numero < numeroSecreto) {
    alert("El numero secreto es mayor");
} else if (numero > numeroSecreto) {
    alert("El numero secreto es menor");
} else {
    alert("Acertaste");
}
```

**Explicación**

`prompt()` obtiene texto. `Number()` intenta convertirlo a número. Las ramas de `if` y `else if` comparan ese valor con el secreto. Este ejemplo supone que ingresas un número válido; la validación llegará más adelante.

**Paso 4.2** Ejecuta la página varias veces e identifica los tres casos: mayor, menor e igual. **Después elimina** el bloque de `prompt()` y `if`: conservaremos únicamente `numeroSecreto` e `intentos`, porque ahora usaremos el campo HTML.

## 5. Tercera etapa: leer el HTML

**Paso 5.1** Localiza en `index.html` los identificadores `numero`, `mensaje` e `intentos`. Observa también el atributo `onclick` de los dos botones.

**Paso 5.2** Debajo de las variables, define la primera función:

**Leer el número desde la interfaz**

```
function comprobar() {
    let numero = document.getElementById("numero").value;
    console.log("El usuario escribio:", numero);
```

```
}
```

**Paso 5.3** Recarga y escribe un valor. Presiona **Comprobar** y revisa la consola.

**Explicación**

El botón contiene `onclick="comprobar()"`, por eso ejecuta la función al hacer clic. `getElementById("numero")` encuentra el campo y `.value` lee su contenido como texto.

**✓ Punto de control**

Comprueba que cada clic imprime el valor actual del campo.

## 6. Cuarta etapa: mostrar pistas en la página

**Paso 6.1** Dentro de `comprobar()`, después de leer el campo, convierte su valor y agrega las condiciones. La función debe quedar así:

**comprobar(): segunda versión**

```
function comprobar() {
    let numero = Number(document.getElementById("numero").value);
    console.log("Intento:", numero);

    if (numero < numeroSecreto) {
        document.getElementById("mensaje").textContent =
            "El numero secreto es mayor.";
    } else if (numero > numeroSecreto) {
        document.getElementById("mensaje").textContent =
            "El numero secreto es menor.";
    } else {
        document.getElementById("mensaje").textContent =
            "Acertaste!";
    }
}
```

**Explicación**

`.textContent` cambia el texto de un elemento del HTML. A diferencia de `alert()`, el mensaje queda visible en la tarjeta. No se modifica el archivo HTML: se modifica lo que muestra el navegador.

**Paso 6.2** Prueba un número inferior y otro superior al secreto. Para esta prueba puedes consultar el valor en la consola.

**✓ Punto de control**

El texto de la página cambia al presionar Comprobar.

## 7. Quinta etapa: contar los intentos

**Paso 7.1** Dentro de `comprobar()`, después de leer el número y **antes** del `if`, inserta:

**Aumentar el contador**

```
intentos = intentos + 1;
```

**Paso 7.2** Después del bloque completo `if / else if / else`, pero **antes de cerrar la función**, añade:

**Actualizar contador en HTML**

```
document.getElementById("intentos").textContent = intentos;
```

**✓ Punto de control**

Cada clic en Comprobar incrementa el número de intentos. Por ahora se contarán incluso entradas vacías: eso se corregirá en la etapa de validación.

## 8. Sexta etapa: comenzar una partida nueva

**Paso 8.1** Debajo de la función `comprobar()`, escribe una segunda función:

**Función nuevoJuego**

```
function nuevoJuego() {
    numeroSecreto = Math.floor(Math.random() * 100) + 1;
    intentos = 0;

    document.getElementById("numero").value = "";
    document.getElementById("mensaje").textContent =
        "Ingresa un numero para comenzar.";
    document.getElementById("intentos").textContent = intentos;

    console.log("Nuevo numero secreto:", numeroSecreto);
}
```

**Explicación**

La variable `numeroSecreto` se declaró fuera de las funciones para que ambas puedan utilizarla. Dentro de `nuevoJuego()` se le asigna otro valor: no debes escribir `let` de nuevo.

**✓ Punto de control**

Después de presionar Nuevo juego, el campo queda vacío, el contador vuelve a cero y se genera otro número.

## 9. Séptima etapa: validar los datos

Una entrada vacía se convierte en cero con `Number("")`; por eso no basta con comparar el número. Tampoco deben contarse valores fuera del intervalo ni textos que no sean números.

**Paso 9.1** Sustituye las primeras líneas de `comprobar()` por este bloque. **Conserva debajo el incremento del contador y las condiciones que ya escribiste.**

**Validación: inicio de comprobar()**

```
function comprobar() {
    let texto = document.getElementById("numero").value;
    let numero = Number(texto);

    if (texto.trim() === "" || !Number.isInteger(numero) ||
        numero < 1 || numero > 100) {
        alert("Escribe un numero entero entre 1 y 100.");
        return;
    }

    // Aqui siguen el contador y las condiciones previas.
```

**Explicación**

`||` significa "o"; `!` niega una condición. `Number.isInteger()` verifica que el valor sea entero y `return` termina la función sin contar un intento inválido. `trim()` permite detectar un campo que solo contiene espacios.

**✓ Punto de control**

Prueba el campo vacío, el valor 0, el 101, un decimal y un entero válido. Solo el entero válido debe incrementar el contador.

## 10. Pruebas y preguntas

1. ¿Qué diferencia hay entre mostrar información con `console.log()`, `alert()` y `textContent`?
2. ¿Por qué `numeroSecreto` e `intentos` deben declararse fuera de `comprobar()`?
3. ¿Qué sucede si se elimina la llamada a `Number()`?
4. ¿Por qué la validación debe realizarse antes de aumentar `intentos`?

Las respuestas se comentarán en clase. Revisa también los casos de prueba del archivo `README.md`.

## 11. Entrega en GitHub

**Paso 11.1** Desde la carpeta local, inicializa el repositorio y registra los archivos:

**Terminal**

```
git init
git add index.html style.css app.js README.md
git commit -m "Practica 01: juego de adivinar el numero"
```

**Paso 11.2** En la **organización de GitHub** indicada por el profesor, crea un repositorio llamado exactamente `p01_js`. De preferencia créalo **vacío**, sin generar otro README, para evitar conflictos.

**Paso 11.3** GitHub mostrará los comandos correspondientes a tu repositorio. Utiliza la dirección SSH que tenga la forma `git@github.com:ORGANIZACION/p01_js.git`:

**Vincular y subir (sustituye ORGANIZACION)**

```
git branch -M main
git remote add origin git@github.com:ORGANIZACION/p01_js.git
git push -u origin main
```

**Explicación**

Usa autenticación SSH, como se ha trabajado en clase. Si haces cambios después de subir, realiza nuevos `git add`, `git commit` y `git push`. Completa tu nombre, grupo, casillas de pruebas y observaciones en el README.

**✓ Punto de control**

Verifica en GitHub que el repositorio `p01_js` incluye los cuatro archivos y que puedes ejecutar el juego desde `index.html` al descargarlo.

## 12. Material inicial proporcionado

Los siguientes tres archivos están completos y se entregan para que no debas programar la estructura o el diseño. El archivo `app.js` se entrega con comentarios iniciales y se desarrolla mediante las etapas anteriores. El `README.md` se entrega también preparado para completar información y pruebas.

### 12.1 Archivo `index.html`

**Contenido proporcionado: index.html**

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Adivina el numero</title>
    <link rel="stylesheet" href="style.css">
    <script src="app.js" defer></script>
</head>
<body>
    <main class="tarjeta">
        <p class="etiqueta">Practica 01 · JavaScript</p>
        <h1>Adivina el numero</h1>
        <p class="descripcion">He pensado un numero entre 1 y 100.
            Intenta descubrirlo con las pistas.</p>

        <label for="numero">Tu intento</label>
        <input type="number" id="numero" min="1" max="100"
               placeholder="Escribe un numero">

        <button type="button" onclick="comprobar()">Comprobar</button>

        <p id="mensaje" class="mensaje" aria-live="polite">
            Ingresa un numero para comenzar.
        </p>
        <p class="contador">Intentos: <strong id="intentos">0</strong></p>

        <button type="button" class="secundario" onclick="nuevoJuego()">
            Nuevo juego
        </button>
    </main>
```

```
</body>
</html>
```

## 12.2 Archivo `style.css`

**Contenido proporcionado: style.css**

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    min-height: 100vh;
    display: grid;
    place-items: center;
    padding: 24px;
    background: #edf3fa;
    color: #19324f;
    font-family: Arial, Helvetica, sans-serif;
}

.tarjeta {
    width: 100%;
    max-width: 440px;
    padding: 34px;
    border-radius: 18px;
    background: white;
    box-shadow: 0 16px 50px rgba(18, 48, 82, 0.12);
}

.etiqueta {
    color: #316eae;
    font-size: 12px;
    font-weight: bold;
    letter-spacing: 1.2px;
    text-transform: uppercase;
}

h1 {
    margin: 12px 0;
    font-size: 30px;
}

.descripcion {
    color: #587089;
    line-height: 1.55;
}

label {
    display: block;
    margin-top: 22px;
    margin-bottom: 9px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 14px;
    border: 1px solid #bdcede;
    border-radius: 9px;
```

```
    font: inherit;
}

button {
    width: 100%;
    margin-top: 16px;
    padding: 13px;
    border: 0;
    border-radius: 9px;
    background: #2165a8;
    color: white;
    font: inherit;
    font-weight: bold;
    cursor: pointer;
}

button:hover {
    background: #174e86;
}

.mensaje {
    min-height: 24px;
    margin-top: 24px;
    text-align: center;
    font-weight: bold;
}

.contador {
    text-align: center;
    color: #587089;
}

.secundario {
    background: #e8f0fa;
    color: #2165a8;
}

.secundario:hover {
    background: #d4e5f6;
}
```

## 12.3 Archivo `README.md`

**Contenido proporcionado: README.md**

```
# Practica 01 - Introduccion a JavaScript

## Datos del estudiante

- Nombre: [Completar]
- Grupo: [Completar]

## Descripcion

Juego en el que el navegador genera un numero secreto entre 1 y 100. El usuario
escribe intentos y recibe pistas hasta adivinarlo. La pagina muestra cuantos
intentos lleva y permite comenzar un juego nuevo.

## Objetivo

Aplicar variables, estructuras de control, funciones, `console.log()`,
```

```
`alert()`, `prompt()`, `Math`, `onclick` y `document.getElementById()`
para desarrollar una pagina web interactiva.

## Archivos

- `index.html`: estructura de la interfaz (proporcionado).
- `style.css`: presentacion de la interfaz (proporcionado).
- `app.js`: comportamiento programado por el estudiante.
- `README.md`: descripcion y evidencia del trabajo.

## Como ejecutar

1. Descargar o clonar el repositorio.
2. Abrir `index.html` en un navegador.
3. Escribir un numero entre 1 y 100 y presionar **Comprobar**.
4. Leer la pista y repetir hasta acertar.
5. Presionar **Nuevo juego** para comenzar otra partida.

## Temas utilizados

Variables, condiciones, funciones, consola, ventanas de aviso,
entrada de datos, generacion de numeros aleatorios y manipulacion del HTML.

## Pruebas realizadas

- [ ] El boton Comprobar muestra pistas correctas.
- [ ] El contador aumenta al ingresar un intento valido.
- [ ] Se detecta el numero correcto.
- [ ] El boton Nuevo juego reinicia el contador y el mensaje.
- [ ] Los valores vacios o fuera de rango no se consideran intentos.

## Observaciones del estudiante

[Escribir brevemente que se aprendio y que dificultades surgieron.]
```